Agentic AI for Enhanced Research

From Idea to Publication with an AI Research Pipeline

Related Tools:

ScholarEngineForAI — `https://github.com/pcwhy/ScholarEngineForAI`

ColabWatcher4aiAgents — `https://github.com/pcwhy/ColabWatcher4aiAgents`

By Dr. Yongxin Liu

Assistant Professor of Data Science, Embry-Riddle Aeronautical University

Associate Editor, Journal of Big Data

Presented at the Dr. Yunpeng (Jack) Zhang Research Group, University of Houston

August 2026

Agenda

1. **Opening** — AI changes how research is *executed*
2. **Key principles** — context windows, mindsets, and layered pipelines
3. **Related Tool I** — Scholar Engine: a multi-stage technical writing skill (+ Phase V for reviewer revisions)
4. **Related Tool II** — Colab Watcher: GPU compute for local AI agents
5. **Synthesis** — one integrated research loop
6. **Takeaway & Q&A**

The core claim

AI does not just change *how we write* research —
it changes **how research is executed**.

But everything starts with a **research question worth answering** —
validated *before* any tooling matters (Scholar Engine Phase I):

Good AI-enhanced research =
research question (what deserves to be studied — the *why*)
+ methodology (how we structure the work)
+ infrastructure (how we get compute)
+ human judgment (final authority).

Part 1

Key Principles for AI-Enhanced Research

The context-window problem

Modern AI systems have a **finite context window**:

- Once the context becomes overloaded, **lossy compression** starts to happen.
- The result: the AI suddenly becomes "*much dumber and much less proactive.*"
- A whole-paper, single-session workflow degrades **silently** — no error message, just worse output.

> *"For a high-quality academic research and writing workflow, current AI systems still do not have a long enough context window."*
> *— ScholarEngineForAI README*

The solution pattern: phases + memo handoff

- Split the research process into **four phases**.
- **End every phase with a memo**: what was done, lessons learned, groundwork for the next phase.
- **Start every new phase in a fresh conversation** — a clean context window.

```
Phase I  → memo → Phase II → memo → Phase III → memo → Phase IV
 (plan)          (build)            (restructure)        (human pass)
```

Each phase runs at full capacity — never compressed.

Engineer vs. scientist mindset — identify the nature of your questions from day 0

Engineer: "*This system can be made to work this way.*"
→ Enough for an EI-indexed paper, possibly an SCI paper.

Scientist: "*Why did other systems fail? Is there a hidden pattern nobody has studied? What does the underlying mathematics say? Where does this system fail, and why?*"
→ The level needed for patents and strong SCI papers.

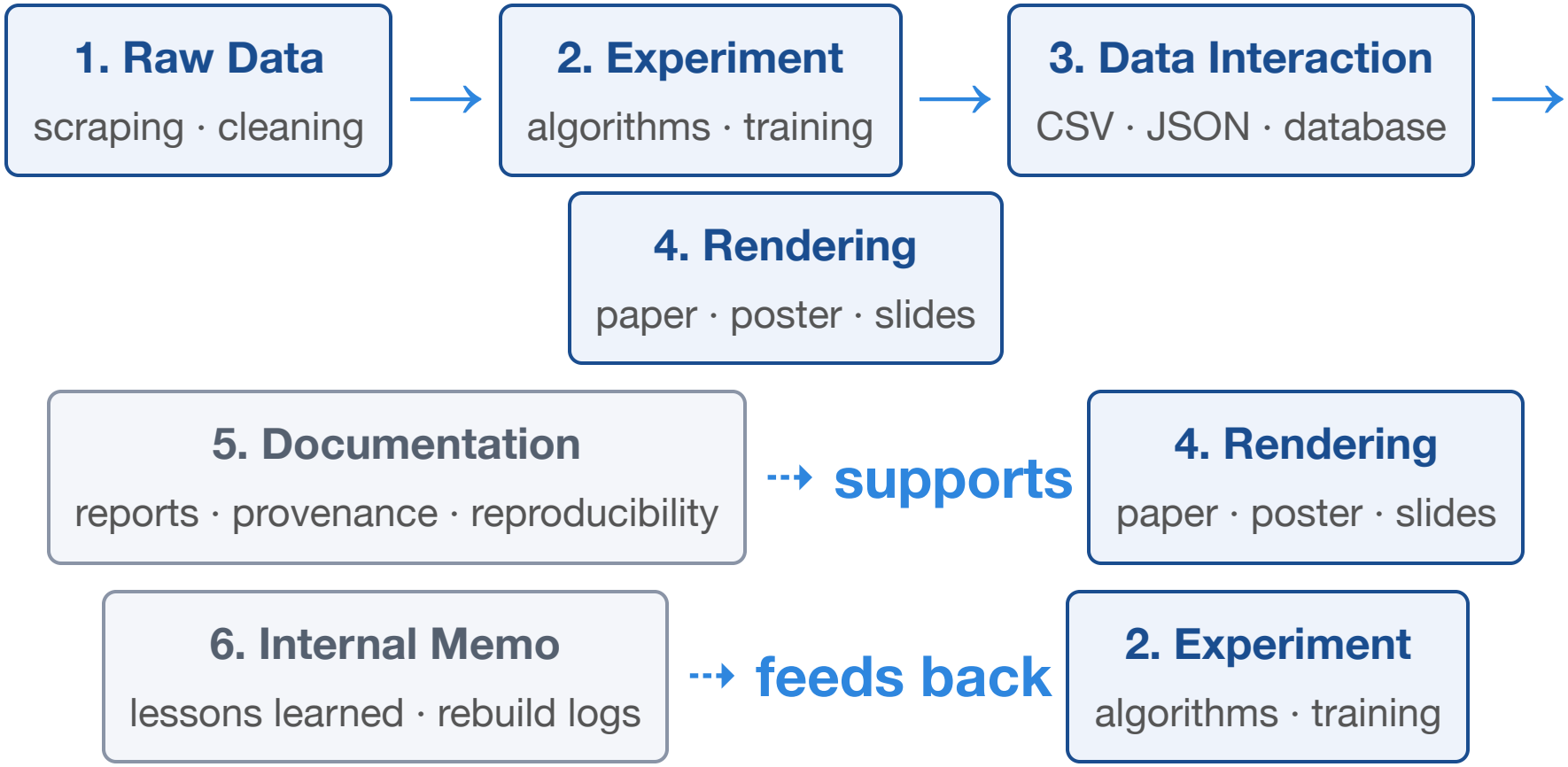
The AI must **reason step by step from facts, verification, and derivation** — and say "I don't know" when uncertain.

Allowing AI to say I'm not sure...

The permission to say "I don't know" is not implicit — it must be written into the prompt:

- explicitly **allow the answer "I don't know"** — otherwise the model will hedge and fabricate instead;
- require every **unverified claim to be flagged as unverified** (or dropped);
- demand each statement be traceable: derived from data, quoted from literature, or admitted as assumption.

The research pipeline from a systems engineering perspective



Every layer gets **its own folder**. No collapsed layers, no entangled logic.

Why layers matter

- **Experiments are separated from LaTeX rendering** — a formatting change never re-triggers simulations.
- The **Data Interaction Layer** (CSV / JSON / databases) is the **single source of truth** for all rendered artifacts.
- Raw-data code (scrapers, crawlers) is preserved with timestamps — reproducible and auditable.
- Two capabilities must always hold:
  - rerun **everything from scratch** and update all data;
  - update all data → regenerate figures/tables → **regenerate the PDF**.

One prompt must never equal an entire paper

Instead of a single "write my whole paper" prompt, we must work in **stages** — precisely because **a single session cannot produce a strong paper**:

- context windows overflow → lossy compression → the AI gets "*much dumber and much less proactive*"
- each phase ends with a **memo**; each new phase starts in a **fresh conversation**
- Phase I plans → Phase II builds & drafts → Phase III restructures → Phase IV human-polishes → Phase V revises

> *Rule of thumb: any prompt or skill that claims to run the whole pipeline in one shot will hand you a class project — not a publication. Split the work, budget the context, and let each session do one job well.*

Part 2

Related Tool I — Scholar Engine: a multi-stage technical writing skill

A skill I generated along the way while using AI to write my own papers.

Scholar Engine: what the repo contains

A **skill package** meant to be installed into an AI agent:

File	Purpose
`README.md`	The four-phase method — *what to do at each stage*
`SKILL.md`	The operating protocol: PI-style writing, pipeline discipline — *how to do each stage well*
`section-protocols.md`	Per-section rubrics: abstract, introduction, methodology, related work, LaTeX guardrails
`openai.yaml`	**UI metadata only** — how the skill appears in the agent's tool picker

> *The `openai.yaml` prompt is interface metadata for **one artifact at a time** — "draft / rewrite / critique **this section**". It is not a recipe for producing a whole paper.*

Read the spec — it protects you

- Read the ScholarEngine norms for academic writing and research **carefully** — a responsibility you owe to yourself.
- Not every open-sourced skill is safe: some contain **malicious instructions** hidden inside.
- A skill is executable guidance for your agent — **audit it before you install it**, just as you would any dependency.

Where we are — Phase I · Plan

1. Phase I · Plan — seed · validate · plan

2. Phase II · Build — implement · draft · document

3. Phase III · Restructure — fix · restructure · story

4. Phase IV · Polish — human final pass

5. Phase V · Revisions — reviewer revisions

*Outcome of this phase: a **validated research question** and a **written plan document**.*

Phase I.1 — Seed the AI with strong papers

- Feed the AI **strong seed papers** using tools such as `arxiv-to-prompt` or `arxiv2md`.
- Two goals in one step:
  i. the AI understands **your research context**;
  ii. the AI learns **your writing style**.
- Skip this and "*the output will read like a class project.*"
- This comes down to one standing habit: **collecting good papers**, continuously.

Phase I.2 — Validate research value, then plan

The **research question is decided here** — before a single line of code or LaTeX is written. If the question fails these tests, no methodology or compute can save it.

Ask the AI to stress-test the idea across five angles:

- **timeliness** — can it ride a current trend?
- **storytelling value** — cross-domain transfer, novel finding, new paradigm?
- **question worth** — scarcity, attention-grabbing value
- **interdisciplinarity** — bring new ideas from other domains
- **theoretical depth** — getting closer to the root cause

Output: a written plan document.

Reference budget: IEEE conference ≈ **≥20 references**; journal ≈ **35**.

Where we are — Phase II · Build

1. Phase I · Plan — seed · validate · plan
2. **Phase II · Build — implement · draft · document**
3. Phase III · Restructure — fix · restructure · story
4. Phase IV · Polish — human final pass
5. Phase V · Revisions — reviewer revisions

Outcome of this phase: ***experiments, figures, and a conference-level full draft*** *— documented and reproducible.*

Phase II.1 — Implement, experiment, draft

- Explore as many combinations and techniques as possible → **sufficient data**.
- **One notebook per attempt / technique** — never a giant all-in-one. A huge notebook is a context and maintenance nightmare: re-running it re-runs everything.
- Figure-producing attempts get **their own notebook** — regenerating figure N must not re-trigger the whole pipeline.
- Notebooks are the lead artifact; no cell exceeds **300 lines**; `.py` files ≤ **400 lines** (SKILL.md: 500).
- **Every program must print progress** — otherwise it times out and the AI "simplifies" your code for you.
- Everything follows **Hungarian notation** — otherwise the codebase becomes unmaintainable.

Phase II.1 — Layer discipline from day one

Split the project immediately into:

```
raw data layer        – scraping, cleaning, ingestion (crawler code preserved)
experiment layer      – algorithms, training, simulation (pure logic)
data interaction layer– raw outputs as CSV/JSON (machine-readable)
rendering layer       – paper, poster, slides (generated only from the layer above)
```

The AI will often try to scrape extra data — keep the crawler code and **label when it was developed**.

Human check required: verify the crawler code actually lives **together with the data** in the raw-data layer — scraper code is exactly the kind of throwaway script that silently gets deleted during "cleanup." Every URL, command, and script used to acquire data must be saved, not just the data itself.

Phase II.1 — Isolate every experiment environment

- **No GPU needed**: have the AI agent create a **virtual environment** or **Docker container** for each experiment — **never install anything directly into your system**.
- **GPU needed**: install nothing locally — hand the job to the **Colab Watcher**, covered in the next section.

Phase II.3 — Let the AI probe the vague problem and bottleneck

- Have the AI inspect **where the system fails**: in which cases does the classifier get things wrong, and under which boundary conditions does the system break?
- Ask whether the failures reveal an **improvement path** — a fixable limitation or a deeper design flaw.
- This is one of AI's biggest strengths: it scans the whole design and helps you locate the **vague problem or bottleneck**.
- **Caveat: the AI's proposed fix is not necessarily correct** — verify it with experiments, never adopt it blindly.

Phase II.2 — The abstract

- One paragraph, at most **one-fifth of a page**.
- Background and significance: at most **4 lines** — the rest goes to method, highlights, results.
- Report only the **two results you are most proud of** (or the two most distinctive findings).
- Self-contained: no internal shorthand or unexplained abbreviations.

Phase II.2 — The five-paragraph Introduction

1. **Background** — big picture, why this matters in general
2. **Gap** — what previous work did, and its common shortcomings
3. **Motivation** — what inspired this paper
4. **Contributions** — an itemized list
5. **Organization** — how the paper is structured

Target length: **1 to 1.5 pages** — not longer.

Phase II.2 — What a contribution really is — the part reviewers and editors read first

- A contribution is **not simply what work you did** — nobody cares about that.
- It is: **what you discovered, what question you answered, where the breakthrough lies**.
- Keep it to **≤ 4 items**.

Phase II.2 — Professional figures & tables

- Figure quality **reflects your paper library** — Nature/Science-caliber style references pay off.
- Use `booktabs` for tables; IEEE styling throughout.
- Generate a **PNG proof for each figure** and inspect it before insertion.
- Consistent **color palette** across all plots.
- **Re-run experiments → update the figures in the paper** — never forget the link.

Phase II.2 — Figure-making discipline

- **Collect good figures all the time** — your library is the style reference and the quality bar.
- **Set hard rules for every figure**: at least **3 curves** per plot, **legible fonts** (never too small), and a **legend** — always.
- After the first successful figure, have the AI **wrap a single unified plotting function** — every later figure inherits the same style.
- **GIS maps**: have the AI **analyze a sample map first** — extract its visual features (layers, colors, symbology) — before drawing anything.

Phase II.2 — Optional: AI-generated overview figure

One high-level figure that captures the whole paper (Gemini Pro + NanoBanana — or current GPT models):

- input: the **full LaTeX** of the paper + **style-reference figures** from your library;
- it works better if you **collect good figures continuously** — your figure library is the reference and sets the visual bar;
- **human inspection is mandatory before use**: check every visual detail and every label — AI diagrams routinely drop details (a plane with a missing wing, garbled text).

Never insert an AI-generated figure unverified.

Phase II.4 — Enforce consistency

- Consistency pass:
  - no **orphan figures/tables** — every asset cited in the text;
  - no invented concepts without human review;
  - remove redundancy and **self-defensive expressions**;
  - define every abbreviation at first use.
- The first-round technical report should already read like an ordinary conference paper.

Phase II.5 — Document code and assets

- Document the code (Markdown, optionally Doxygen).
- For **every figure and table**: what it contains, what it is for, which code generated it.
- Verify two capabilities end-to-end:
  i. **rerun everything from scratch** and update all data;
  ii. update data → regenerate figures/tables → **regenerate the paper PDF**.

Where we are — Phase IV · Polish

1. Phase I · Plan — seed · validate · plan
2. Phase II · Build — implement · draft · document
3. Phase III · Restructure — fix · restructure · story
4. **Phase IV · Polish — human final pass**
5. Phase V · Revisions — reviewer revisions

*Outcome of this phase: a **polished manuscript** — fake references removed, hallucinations cleaned, human touch added.*

Phase IV — Human final pass

- A human reads the paper carefully and polishes it.
- Kill **fake references** with tools like **Sourcely, Paperpal, Elicit**.
- Remove repetitive wording and **hallucinations**.
- Add the **real human touch** — the part no AI can replace.

Where we are — Phase V · Revisions

1. Phase I · Plan — seed · validate · plan
2. Phase II · Build — implement · draft · document
3. Phase III · Restructure — fix · restructure · story
4. Phase IV · Polish — human final pass
5. **Phase V · Revisions — reviewer revisions**

*Outcome of this phase: a **clean revision** plus a **synchronized point-by-point response letter**.*

Phase V — Reviewer revisions: the workflow

When reviewer comments come back, treat the revision as a **new phase in a fresh context** — not a patch on the old session:

- **(a) Per-reviewer intake** — put each reviewer's comments into a **separate file**; in a **new session**, have the AI read your submitted manuscript and the comment files — then **only acknowledge** having read them, nothing more.
- **(b) Plan before touching the manuscript** — have the AI **classify the comments** and produce a **phased revision plan, including any supplementary experiments**. **You review and approve the plan first.**
- Then revise phase by phase, each in a fresh session, keeping phase-by-phase revision notes with an explicit *Reviewer Comments Addressed* section.

Phase V — The response letter

- **One Response Letter per reviewer**, drafted by the authors themselves — never silently delegated to the AI.
- The final letter must state: **what experiments were run, how the manuscript was changed, and exactly where** each change appears in the resubmission.
- Write it in **LaTeX** — it signals professionalism. Keep the language **plain and easy to read**: the AI-internal jargon Codex loves to write is glaring to reviewers and must **never** appear.
- Tone: professional, respectful, **non-defensive** — acknowledge real weaknesses *before* explaining the repair; repair the manuscript and the evidence chain, and revise the claim if the result no longer supports it.

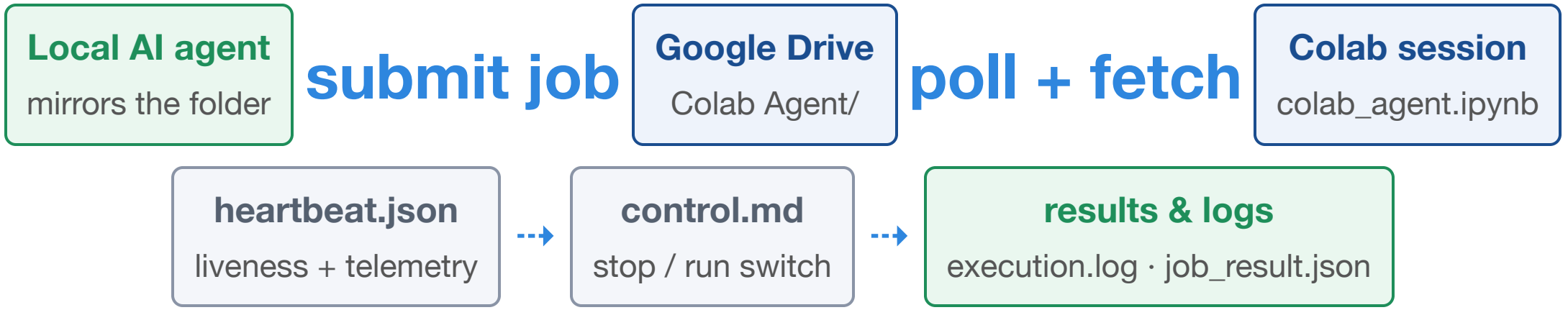
Part 3

Related Tool II — Colab Watcher: GPU compute for AI agents

The compute problem

- Writing and planning are cheap — **training and heavy experiments are not**.
- Local AI agents are capable but **CPU-bound**.
- Google Colab offers free/cheap **GPUs** — but sessions are ephemeral and manual.
- **ColabWatcher4aiAgents** gives agents remote compute through a **Drive folder**: no servers, no API keys, no SSH — just a file queue.

Architecture: Drive as the message bus



No servers, no API keys — the notebook runs `watch_forever()` and the agent works through a **local Drive mirror**.

Setup first: mirror your Drive to the local disk

The local agent never calls Google APIs — it reads and writes through your **local mirror of Drive**:

1. Install **Google Drive for Desktop** and sign in with the same account that runs the Colab session;
2. Mirror the `Colab Agent/` folder so it appears as a normal local folder;
3. **Tell the AI agent where the mirror lives** locally, and let it read the README / SKILL.md;
4. The agent drops jobs into the local mirror → Drive syncs → the Colab watcher picks them up → results flow back into `done/`.

Both sides see the same folder — the filesystem is the shared bus.

Drive layout: a file-based job queue

```
Colab Agent/
  control.md          ← operator switch ("stop" / "run")
  heartbeat.json      ← liveness + telemetry
  jobs/
    inbox/            ← submit jobs here
    running/          ← executing (do not touch)
    done/             ← finished workspaces
    failed/           ← failures and stops
```

No database. The **filesystem is the state**.

Job types

Type	How it runs
single `.py`	executed directly
single `.ipynb`	executed through **papermill**
bundled folder	`job.json` declares the `entrypoint`

```
{ "entrypoint": "main.py" }          // or "notebooks/run.ipynb"
```

Safety rule: the entrypoint **must stay inside the job folder** (path-traversal guard).

One runnable file → auto-detected; multiple → `job.json` required.

The watcher loop

```
while True:
    write_heartbeat()                  # every 30 s
    if stop_requested(): sleep(); continue
    for job in list_pending_jobs():
        run_in_isolated_workspace() # timeout 30 min
```

- polls every **15 s**; heartbeat every **30 s**; job timeout **30 min**
- each job runs in its own workspace under `jobs/running/<job_id>/`
- **stop** (`control.md`): terminates the active job, but the watcher **stays alive**
- **spread different tasks across separate notebooks / runtimes** — if one Colab runtime dies, the others keep running; a hung session otherwise drags every job down with it

Telemetry & control

`heartbeat.json` reports runtime health:

```
{
  "timestamp_utc": "2026-08-10T12:00:00Z",
  "state": "watching",
  "control_stop_requested": false,
  "pending_jobs": 2,
  "runtime": {
    "memory": {"used_bytes": ..., "usage_percent": ...},
    "disk_free_bytes": ...,
    "gpu": {"devices": [{"name": "T4", "utilization_gpu_percent": 87}]}
  }
}
```

Per job: `execution.log` + `job_result.json` → status `done` / `failed` / `stopped`.

GPU reality check

- A **GPU-backed Colab runtime is necessary but not sufficient**.
- `papermill` does **not** enable GPU — it just runs the notebook in the current runtime.
- The **job code itself** must use the GPU:
  - PyTorch: `torch.cuda.is_available()` + move tensors to CUDA explicitly;
  - TensorFlow: normal GPU-aware configuration.
- Lesson: infrastructure provides *capacity*; the experiment provides *usage*.

Test case: `ThreeLayerNN_Pass.ipynb`

A three-layer MLP on MNIST (784 → 30 → 20 → 10) from ERAU's **DS615 Data Modeling** course.

Research idea: visualize **"pass patterns"** —

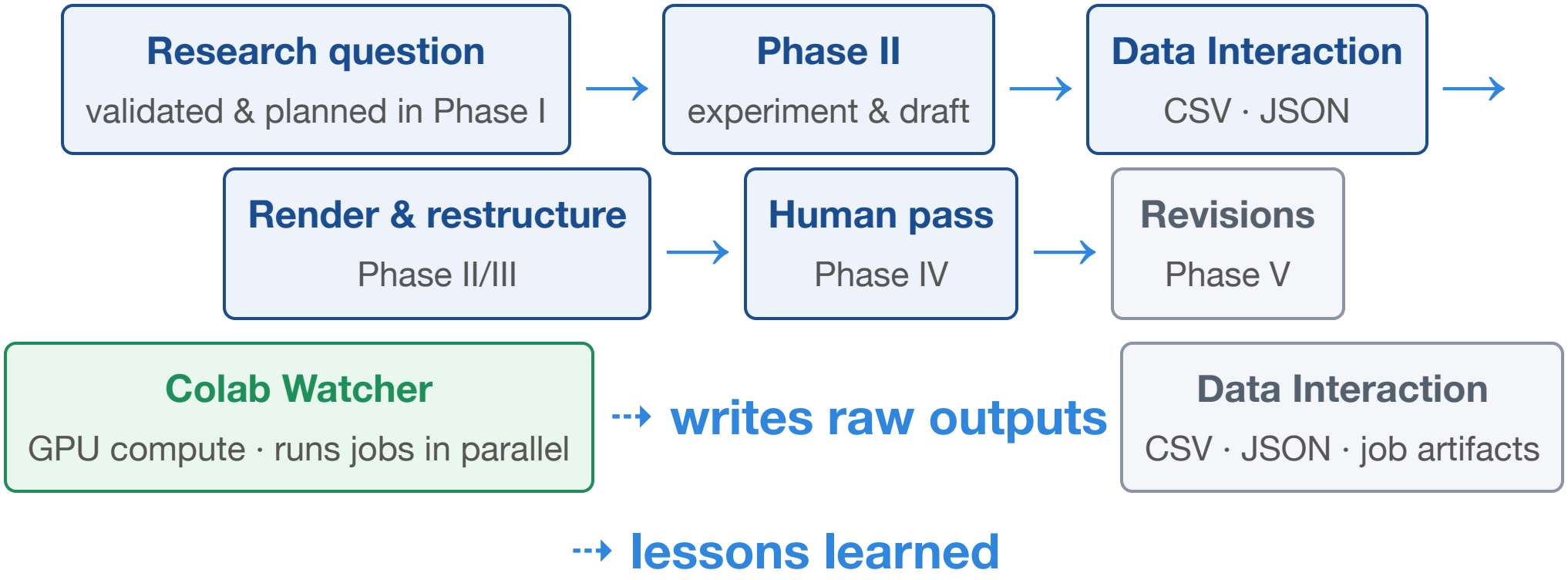
- ReLU-activated weight rows, reshaped to **28×28 images**;
- **blended patterns** via matrix products across layers — what features "pass" through the network.

Exactly the kind of GPU job a local agent can queue, run, and collect from Drive.

Part 4

One integrated research loop

The complete loop



AI executes the loop; the human is the PI — planning sign-off and final judgment.

AI-era rule: publications are systems

- Treat **any AI-assisted output intended for publication** as a **product development pipeline with phased quality control** — Phases I–V are its quality gates.
- In the AI era, **everyone must become a systems engineer**: you design the pipeline, its checkpoints, and where human judgment gates the flow.

Transferable lessons

- **Budget the context** — phases + memo handoffs, never one giant session.
- **Decouple layers** — data, experiment, rendering, documentation, memos.
- **Make everything observable** — heartbeats, progress prints, execution logs.
- **Package skills as institutional knowledge** — a skill file turns any session into an expert. Ask the AI to write skills before you leave for a coffee.
- **Put the human at the right moments** — idea validation, quality control, and the final pass.

Hands-on (suggested exercise)

The point of this exercise: **the AI agent submits the jobs and retrieves the results — you supervise**:

1. Open `colab_agent.ipynb` in Colab with a **GPU runtime**; mount Drive; run `watch_forever()`.
2. Mirror the `Colab Agent/` folder locally, and tell the agent where it lives.
3. Ask the agent to prepare an experiment, **submit it** as a bundled job into `inbox/`, then **retrieve the results** from `done/` once the watcher finishes.
4. Ask the agent to watch `heartbeat.json` and stop jobs via `control.md`, then draft one paper section under **Scholar Engine** rules (PI voice, five-paragraph intro, no hype).

Takeaway

AI-enhanced research is a **pipeline discipline** — and it starts with a question:

Ask the right question · Structure the work · Budget the context · Rent the compute · Keep the human as PI.

Repos: `https://github.com/pcwhy/ScholarEngineForAI` · `https://github.com/pcwhy/ColabWatcher4aiAgents`

Resources

Purpose	Tools
Seed papers → prompts	`arxiv-to-prompt`, `arxiv2md`
Notebook execution	`papermill`
Reference checking	Sourcely, Paperpal, Elicit
Overview figures	Gemini Pro + NanoBanana
Skill packaging	`SKILL.md` + `openai.yaml` (+ Codex/agents)

Questions?